Detailed Version of 3.0 Methodology, JDehghani 201725400

3.0 Methodology

This chapter details the three-stage computational pipeline developed to generate, preprocess and evaluate the tree-based ML surrogate models. The three-stage pipeline followed by an evaluation of results is visualised in Fig. 1. To maintain project management, the Project Risk Register, provided in App. B, was consistently monitored and updated throughout the process of this methodology. To ensure full reproducibility, the full project package and user guides are provided in App. A.

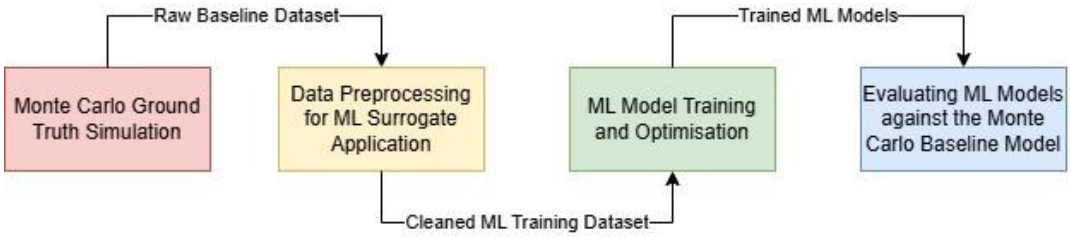

Figure 1: Project Approach Flowchart.

3.1 Experimental Design & Apparatus

To resolve the computational and regulatory bottlenecks established in Ch. 2, the methodology was structured as a three-stage pipeline: 1) Stochastic base-data generation, 2) Data Preprocessing, 3) Surrogate model training and validation.

3.1.1 Physical Apparatus

The computation for both the stochastic (OpenMC) model and the surrogate (RFR) model was executed locally on a PC with the specifications detailed in Table 1.

Table 1: Local Hardware Specifications.

Type	Specification
CPU	Intel i7-1260P (12 cores, 16 threads, 2.1 GHz)
RAM	8 GB
Operating System	Windows 11/ Windows Subsystem for Linux 2 (WSL2) via Ubuntu

Both the traditional MC simulation and the ML surrogate models were run on the exact same machine. Since the hardware is kept constant, any large difference in computation time is

undeniably due to the efficiency of the ML algorithm, establishing a controlled baseline for computational cost. OpenMC natively utilises OpenMP multithreading (Romano *et al.*, 2015); automatically using all 16 available CPU threads. Despite maximising hardware utilisation, the Monte Carlo method still proved too computationally expensive for real-time application (described further in Sec. 3.2.6).

3.1.2 Software Stack

All software versions were strictly managed to guarantee the reproducibility of this methodology. A Conda Environment [Version 26.1.0] running the latest stable version of python [Version 3.14.4] was used to manage computation across all three pipeline stages.

Data Generation:

- **WSL2 via Ubuntu:** OpenMC relies heavily on native POSIX systems, unavailable on Windows. Therefore, WSL2 is a requirement for running Linux directly within windows.
- **OpenMC [Version 0.15.3] and ENDF/B-VII.1:** OpenMC (Romano *et al.*, 2015) provides the industry-standard for simulating Monte Carlo models, while the ENDF/B-VII.1 library (Chadwick *et al.*, 2011) provides highly verified nuclear cross-sections which are necessary for OpenMC to simulate accurate neutron transport.
- **OpenMC Python API [version 0.15.3]:** The base reactor simulation was created using this API, this allows python to build the reactor geometry and communicate with OpenMC to run the Monte Carlo simulations.
- **MIT ExaSMR benchmark:** this package is MIT's ExaSMR benchmark (Romano), providing a verified NuScale-like model for OpenMC simulation.
- **SciPy [Version 1.17.1]:** The Scipy.stats.qmc module provided the Latin Hypercube sampling function, ensuring a diverse range of features (Virtanen *et al.*, 2020).
- **IAPWS:** This library dynamically calculates water density based on temperature, ensuring the thermodynamic properties injected into the reactor model obey the laws of physics (Kretzschmar and Wagner, 2019).

Data preprocessing:

- **Jupyter [Version 7.5.5]:** A jupyter notebook (within a python base kernal) was used, providing the standard tool for data cleaning and results analysis.
- **Matplotlib [Version 3.10] and Seaborn [Version 0.13.2]:** These libraries were used to generate visual plots and graphs required to validate and assess simulation results.

Machine Learning Models:

- **Scikit-learn [Version 1.8]:** This was the foundational machine learning library used to build, train and test the ML surrogate models. It also provides metrics for uncertainty (via the K-fold module) and feature importance.
- **XGBoost [Version 3.2.0]:** This library contains the Chen *et al.* (2019) XGBoosting algorithm for comparative testing against the traditional RFR architecture.
- **Joblib [Version 1.5.3]:** This library was used to export the fully trained surrogate models to a `.pkl` file, for instant use in testing without the need for retraining.
- **Matplotlib [Version 3.10] and Seaborn [Version 0.13.2]:** Used to generate visual plots for results and analysis.

3.2 High-Fidelity Stochastic Base-Data Generation

The primary objective of this phase was to generate a robust ground-truth dataset that represents the complex operational parameters of a commercial SMR. This section details the baseline geometry, the selected input features and the automated OpenMC pipeline utilised to extract the target variable (k_{eff}).

3.2.1 Base Model

The base OpenMC model used for this methodology was the MIT ExaSMR benchmark, an open-source 3D model mimicking the design of the NuScale power module (NuScale Power LLC, 2016). This package consists of a group of python scripts (importantly `build-core-fresh.py` and the `/smr` folder) that use OpenMCs python API to construct the geometry of a baseline core. Since the exact NuScale geometric and material properties are proprietary (strictly controlled by the NRC), using an exact replica of a commercial SMR is impossible. However, the MIT model provides a pre-verified replica intended on capturing most of the physical complexities involved in modelling a full reactor core. This isolates the subsequent ML surrogate model from foundational geometric errors within the OpenMC model.

To compensate for proprietary data, the MIT ExaSMR model relies on geometric and material approximations. For example, specific NuScale fuel assembly details are approximated, and control rod insertion step heights are derived from a separate MIT reactor design; the Westinghouse BEAVRS benchmark (Horelik *et al.*, 2013). Additionally, the model assumes a

Beginning of Life (BOL) fuel composition. Despite these limitations, the computational burden and physical complexity of the MIT model remain identical to reality, making it a suitable model for this application.

The ExaSMR benchmark acts as a static baseline model. To utilise it for generating a dataset, three core changes had to be made:

- The original model defined the geometry and material properties of control rods but left all guide tubes empty in its assemblies. The scripts `core.py` and `surfaces.py` in the `/smr` folder were altered to physically load control rods into the OpenMC assembly (detailed further in Sec. 3.2.3).
- The OpenMC python API undergoes frequent updates effecting it notation. Since the open-source model was last committed in 2022, depreciated API code (such as `rectangular_prism` and `source.Source`) required updating to remain compatible with modern versions (OpenMC Development Team, 2024).
- To create physical alterations within the reactor model (necessary for having a diverse dataset for the surrogate model), a pair of code injection blocks were implemented into the base `build-core-fresh.py` script. These injections allowed the baseline model to be overwritten with the chosen input parameters (detailed in Sec. 3.2.2).

3.2.2 Input Parameters and boundaries

To capture a varied dataset, specific input parameters (summarised in Table 2) where selected to manipulate k_{eff}. To ensure the model remained physically viable, the input boundaries where specifically chosen based on the nominal operating conditions of the NuScale power module.

Table 2: Input Parameter Boundaries and Influence on k_{eff}.

Input Feature	**Boundaries**		**Effect on k_{eff} via the 6 Factor Formula**
	Lower	**Upper**	
Fuel temperature	500 K	1200 K	Increased fuel temperature reduces resonance escape probability (p).
Moderator temperature	500 K	600 K	Increasing moderator temperature decreases density.
Moderator Density *	0.65 g/cm^3	0.76 g/cm^3	Increased moderator density increases p and simultaneously decreases thermal utilisation (f).
Cladding Temperature	500 K	700 K	Negligible impact on k_{eff}.

Boron Concentration	0.01 mg/L (ppm)	1235 mg/L (ppm)	Increasing boron concentration lowers the thermal utilisation factor (f).
Control rod step	0 (ARI)	248 (ARO)	Thermal shadowing from control rods reduces k_{eff}.

** Not independently sampled; calculated dynamically based on Moderator Temperature at 12.76 MPa.*

Increasing Fuel Temperature directly drives the Doppler effect, where U-238 resonances widen, capturing more neutrons and therefore reducing k_{eff} (Mercier, 2025, Ch. 15). The fuel temperature range of 500-1200K captures the temperature of the core from hot shutdown (488 K, conservative as fuel will always be the hottest part of the system) (NuScale Power LLC, 2016, Table 4.3-5) to beyond the peak fuel pellet temperature of 1155 K (NuScale Power LLC, 2016, Table 4.4-8).

The NPM utilises a combined coolant and moderator system using borated water. Increasing Moderator Temperature causes the fluid to expand, reducing the concentration of moderating atoms, trapping fast moving neutrons in the unreactive "epithermal range" (Mercier, 2025, Ch. 4). The moderator temperature reflects a lower boundary of NuScale's core temperature at hot shutdown (488 K) to slightly above the high temperature limit of 595 K (NuScale Power LLC, 2016, Fig. 4.4-9). The upper boundary is conservatively below the boiling point of water at NuScale's nominal system pressure of 12.76 MPa (NuScale Power LLC, 2016, Table 4.1-1) to accurately reflect operating conditions. The Moderator Density was calculated dynamically before each simulation based on the Moderator Temperature at NuScale's nominal system pressure (which was verified to be the system pressure of the MIT model via `__init__.py`). Including moderator density as a dependent input ensures the model reflects reality.

The NPM's cladding is composed of a M5 alloy (M5) and Zirconium-4 (Zry-4). The Cladding Temperature lower limit is the hot shutdown temperature and extends conservatively above the 595 K high temperature limit (NuScale Power LLC, 2016, Fig. 4.4-9). Importantly, the 700 K upper boundary remains below the phase transformation temperature of both M5 and Zry-4, which is around 1000 K (Kecek A *et al.*, 2016, Table 1).

Boron is the soluble material dissolved in the moderator to absorb thermal neutrons, serving as the primary long-term mechanism to control reactivity. The 1235ppm upper limit matches NuScale's initial Boron Concentration at BOL (NuScale Power LLC, 2016, Sec. 4.3.2.1). As the fuel material depletes, the Boron Concentration typically reaches 20 ppm or less (NuScale Power LLC, 2016, Sec. 4.3.1.1); therefore, the lower boundary is set at 0.01 to prevent a divide-by-zero error in OpenMC's mathematical process.

Alongside boron concentration, control rods (CR) act as the primary method of controlling core reactivity (NuScale Power LLC, 2016, Sec. 4.1.1). CRs introduce negative reactivity into the reactor core, absorbing neutrons to control k_{eff} in real time (known as neutron shadowing). The NPM organises its control rods into two groups: the regulating bank and the shutdown bank. The shutdown banks remain in an All Rods Out (ARO) state while the regulating bank is moved in steps to replicate real-world NuScale operation (NuScale Power LLC, 2016, Sec. 4.3.2.1). The MIT model assigns Bank D control rods as the regulating bank and the other banks A, B and C as shutdown banks. Additionally, The MIT model's control rod insertion is split into 248 steps at approximately 1.58 cm step width, derived from MIT's Westinghouse BEAVRS benchmark to estimate the proprietary control rod insertion stages of the NPM (explained further in Sec. 3.2.1).

3.2.3 Updating the Core Builder Script (Code Injections)

Auditing the `materials.py`, `surfaces.py` and `core.py` scripts in the `/smr` folder provided the specific names used for each material in MIT's benchmark model. The second code injection into the core builder script utilises these material names by iterating through each material (such as Zircaloy-4), recognising each relevant material with substring matching to ensure it captures batch materials (such as different fuel pin enrichments). After recognising the specific material, its property is updated with the input parameter (discussed in Sec. 3.2.2) using OpenMC API (OpenMC Development Team, 2024) object modification (e.g. `mat.temperature`). This second code injection was implemented in the core builder script (`build-core-fresh.py`) after the OpenMC geometry is created but before it is saved to .xml files, catching it at the perfect time to append its material properties.

The MIT code inputs boron concentration as an atomic fraction. The `materials.py` script provides the mathematical process of calculating this using OpenMC's `atomic_weight` API. Before the boron concentration is updated in OpenMC, `mat._nuclides.clear()` wipes the existing baseline nuclides to make sure they don't layer on top of the newly calculated nuclides in the final model (OpenMC Development Team, 2024).

The audit of the `/smr` folder revealed that despite the MIT script including all relevant materials and geometries for the control rods, they were absent from the assemblies in the OpenMC model. The regulating CR bank D (described in Sec. 3.2.2) had to be added manually to the model assembly in `assemblies.py`. Control rod insertion had to be moved to the first injection block to ensure CR insertion was altered before the OpenMC geometry is built. Reading the sampled step

from the input file, the insertion of the control rods was changed by altering the plane of the bottom of the rods (Z-plane) to change how far they reach into the reactor core:

$$Z_{bottom} = Z_{fully\ inserted} + (Control\ rod\ step\ \times Step\ size) \quad (6)$$

$$Z_{top} = Z_{bottom} + Control\ rod\ length \quad (7)$$

As the control rod step decreases, the Z-Plane boundaries of the control rods translate into the core, acting as the insertion of the bank D control rods. The use of a 1D model for CR insertion is verified in the literature (Mercier, 2025, p. 56). Furthermore, the CR guide tube dashpots were flattened to the bottom of the fuel rod to ensure that surface crossover errors do not occur when fully inserted rods touch the boundary at the end of the guide tubes (OpenMC Development Team, 2024).

3.2.4 Sampling Strategy

To gather a diverse combination of input variables, a Latin Hypercube sampler (LHS) was chosen over a simple random number generator, such as `np.random`. The LHS divides each parameter's range into non-overlapping intervals to prevent clustering (Virtanen *et al.*, 2020). This ultimately limits the number of simulations required to train a surrogate model, without sacrificing the quality of data (Alizadeh, Allen and Mistree, 2020).

The raw LHS matrix (between 1 and 0) was scaled to the upper and lower parameter boundaries to create a single matrix storing every sample of inputs. These were then unpacked into the individual variable arrays to be applied to the Monte Carlo model. The moderator density wasn't included in the LHS because it is calculated during each iteration based on moderator temperature. Although the temperatures and boron concentration are continuous numbers, the control rod steps are discrete. The continuous LHS outputs for CR steps were rounded (`np.round` (Harris *et al.*, 2020) and `.astype(int)`) to an integer value, ensuring that the control rods are not positioned mid-step.

3.2.5 Automated Pipeline Architecture

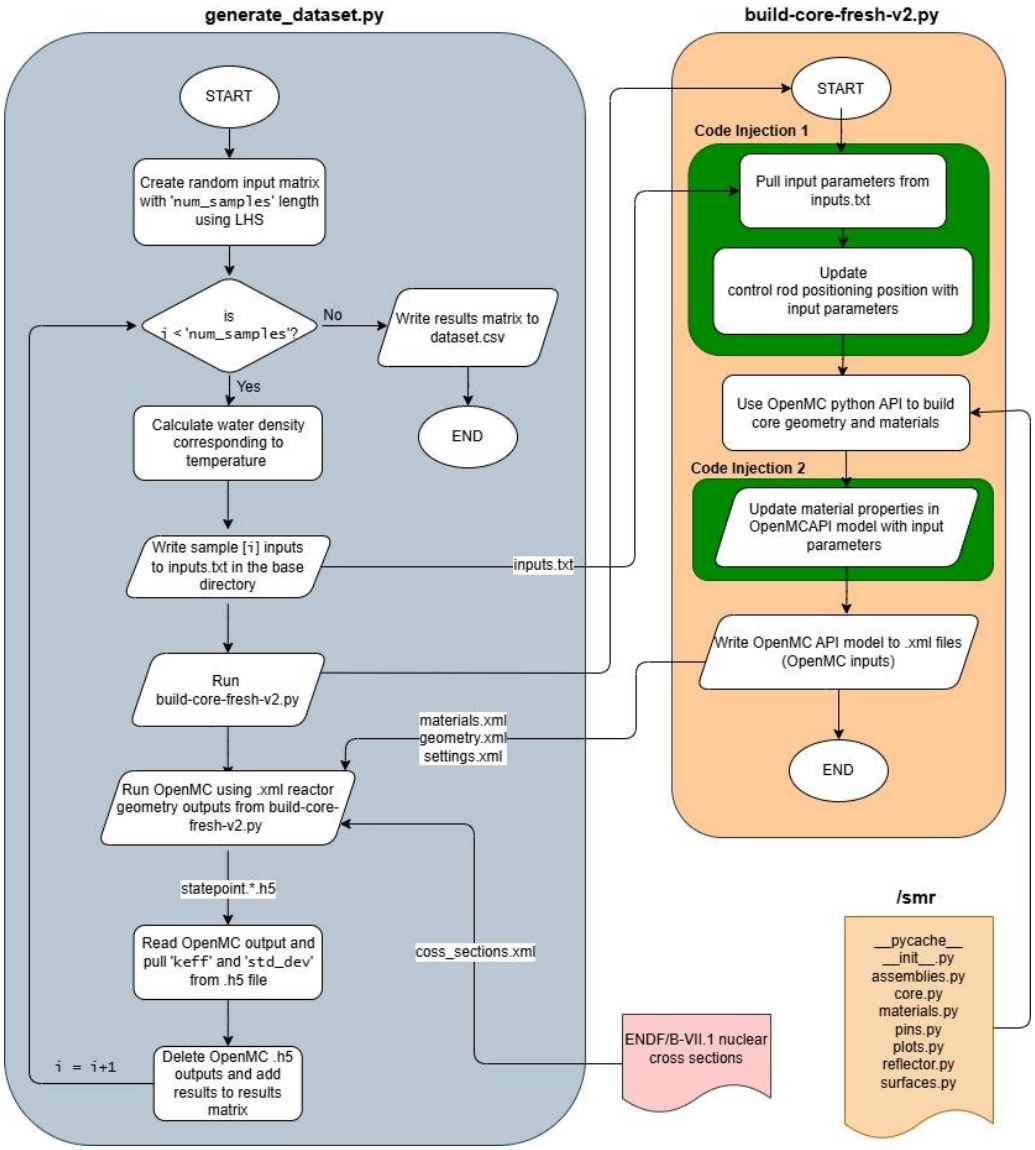

Figure 2: Architecture Diagram of the Automated Dataset Generation Pipeline, including all file interactions.

As illustrated in Fig. 1, the data generation pipeline is governed by the overarching script, `generate_dataset.py`. This script initiates the LHS input matrix, then runs an automated loop that performs the following for each sample:

- Calculates moderator density using the IAPWS library (described in Sec. 3.2.2).
- Writes the current sample's parameters to 'inputs.txt' so they can be read by `build-core-fresh-v2.py` to update the model (described in Sec. 3.2.3).
- Runs `build-core-fresh-v2.py`.
    - Code injection 1 reads the input file and alters control rod insertion.
    - The MIT script uses the OpenMC python API to build the reactor geometry.
    - Code injection 2 alters the material properties of the model based on the input values.

    - outputs .xml geometry files to a new folder `/core-fresh`.
- Uses the .xml files and ENDF/B-VII.1 nuclear cross sections to run an OpenMC simulation.
- Pulls k_{eff} and its standard deviation from the OpenMC output and stores them in a matrix. The standard deviation acts as a metric to quantify the fidelity of results.
- Deletes the OpenMC output files from storage, preventing the program from stacking large files as the loop iterates.

Once done, the overarching script reformats the results matrix, using the pandas library (McKinney, 2010) and saves them to a CSV dataset. To ensure a robust design, the loop is wrapped in a `try/except/finally` block, guaranteeing that the results are always saved if interrupted. Additionally, `if/continue` blocks were used throughout the script to skip possible errors (such as OpenMC failing to converge on k_{eff}), ensuring the automatic loop doesn't fail mid run. Another measure to ensure the loop was not interrupted was by running `generate_dataset.py` with the Linux command `nohup` (no hang up), which continues to run the script if the computer sleeps or the terminal closes.

3.2.6 Simulation Parameter Tuning

A dataset of 1,000 samples was targeted to train the ML models. While Tao *et al.* (2025) notes that 90% of material science ML studies use datasets of <1000 samples, this is on the upper boundary of what is considered small, necessitating careful hyperparameter tuning to avoid overfitting, particularly for the XGBoost model.

Table 3: Simulation Parameters and their Effects on Fidelity and Computation.

Parameter	Chosen Value	Primary Impact
Number of Samples	1,000	The dataset length required for training a reliable ML surrogate.
Simulated Particles	10,000 / batch	Primary driver for k_{eff} standard deviation.
Active Batches	200	Same effect on k_{eff} as particles but requires significantly more computation.
Inactive Batches	50	Ensures that the simulation converges.
Annular Rings	1	Eliminates redundant models for burnup calculations.
Axial Subdivisions	5	Retains sufficient control rod resolution while reducing computation.

Since the standard deviation regresses proportional to $1/\sqrt{N}$, the number of active batches and simulated particles are the primary drivers for the fidelity of the model. While increasing either

parameter has the same effect on standard deviation, increasing batches is far more computationally expensive than simulating more particles (Romano *et al.*, 2015). A minimum of 10,000 particles per batch was selected following similar studies (Barrientos and Alfonso, 2021). 200 active batches was paired with this to produce an acceptable standard deviation of less than 1% (Gill, Griesheimer and Aumiller, 2017). Furthermore, inactive batches are required to ensure the model fully converges before it starts collecting data (Barrientos and Alfonso, 2021). A similar study (Barrientos and Alfonso, 2021) successfully converged using 50 inactive batches, verifying its adequacy for this model.

The original MIT ExaSMR benchmark modelled the fuel pins with 10 annular rings and 196 axial subdivisions because it was intended for long-term depletion (burnup) calculations. However, the scope of this study relies only on a fresh core, meaning these parameters significantly increase computation time while having very little effect on k_{eff}. Additionally, axial subdivisions do affect the geometry of the control rods. Using 5 subdivisions provides sufficient resolution to capture the neutron shadowing effect introduced by inserting the control rods.

3.3 Data Preprocessing

3.3.1 Data Cleaning and Validating Monte Carlo Results

The raw dataset produced by the previous stage was processed within a Jupyter notebook to generate a clean input for the surrogate model. First, any samples containing null values for k_{eff} (representing failed OpenMC runs) were removed using the Pandas library, ensuring the surrogate model is trained only on the ground-truth results. Compute time is also extracted to establish a baseline for Sec. 4.4. Second, the non-physical parameters (standard deviation and compute time) were excluded from the dataset. A Spearman correlation heatmap (Fig. 2) was generated using `matplotlib` to visually assess whether the relationships between the parameters in the raw dataset align with physics (Shaqiri *et al.*, 2023).

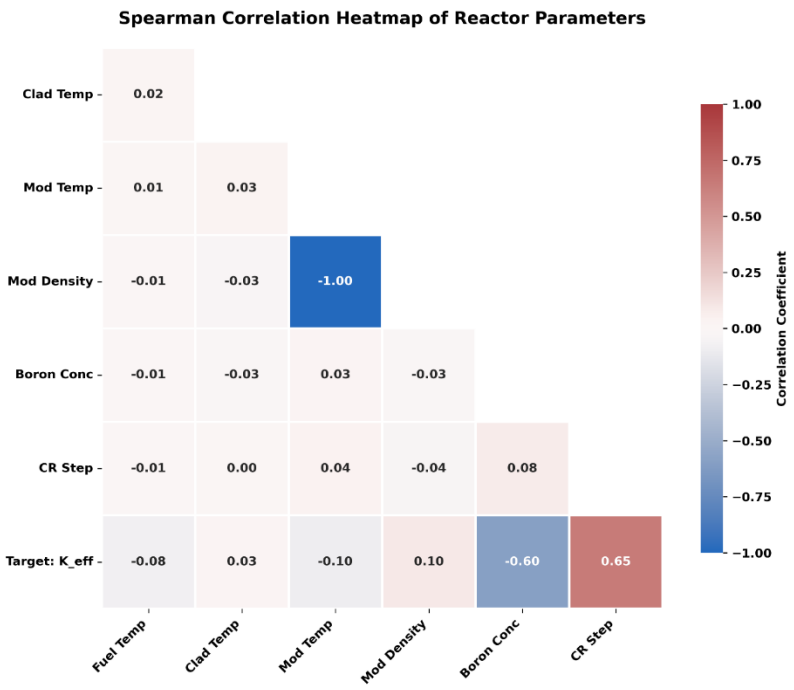

Figure 3: Spearman Correlation Heatmap of Physical Reactor Parameters.

An analysis of the correlation matrix reveals that the interactions between the input features are near zero. This is a good indication that the Latin Hypercube Sampler worked effectively, generating a range of non-clustered (independent) samples. This eliminates the issue of unintended biases within the random input dataset that can cause overfitting, especially with boosting ML architectures (Breiman, 2001). The strongest correlating features with the target (k_{eff}) are control rod step (0.65 correlation) and boron concentration (-0.60 correlation). This aligns with the underlying physics that was simulated by the stochastic model, where these features are the primary methods of affecting reactor criticality. Additionally, moderator temperature and density have a near-perfect correlation (-1.00 correlation); this is expected since the MC model uses the IAPWS library to dynamically calculate moderator density based on its temperature with a fixed system pressure.

Since moderator temperature and density have a near-perfect correlation, using both as features for the surrogate model will have no effect on the accuracy of the model. However, when measuring feature importance, the near-perfect correlation will mask the true importance of these variables (Breiman, 2001), complicating the interpretability as well as adding redundant processing to the ML models. For these reasons, moderator density is excluded from the cleaned dataset alongside the non-physical parameters.

3.4 Surrogate Model Architecture and Training

3.4.1 ML Algorithm

Due to the computational limitations of generating a high-fidelity dataset, the ML models' sample size was limited to 1000 (justified in Sec. 3.2.6), this is classed as a small dataset (Tao *et al.*, 2025). Tao *et al.* (2025) explains that overly complex algorithms like NNs are prone to overfitting on small sample sizes due to their excessive complexity, using a large number of parameters. In contrast to this, tree-based learning algorithms like the RFR (Breiman, 2001) reduce overall model bias and variance (overfitting) through the collaboration of multiple ML models, making them far better suited to small datasets.

Decision tree architectures are inherently "intuitive and the results are interpretable. Moreover, compared to other algorithms, decision trees require less effort for data preparation, and do not require data normalization and scaling" (Racheal, Liu and Ayodeji, 2022, p. 2). This avoids the NNs major limitation of the "black box dilemma" (Huang *et al.*, 2023, p. 10), providing the required explainability to satisfy ALARP.

While the RFR serves as an effective base model to prove the transparency and speed of a tree-based ML model, its predictive accuracy is not as strong as modern ML architectures like eXtreme Gradient Boosting (Chen and Guestrin, 2016). Like the RFR, XGBoost is a supervised learning method based on decision trees, providing the same transparency and reduction in model bias. XGBoost builds trees sequentially, where each new tree learns from the error of previous trees, correcting areas where the previous model underperformed (Bisong, 2019). This makes XGBoost highly effective for achieving a high level of accuracy on complex tabular data (like the physical feature values).

Both tree-based algorithms were programmed with identical scripts (except hyperparameters) using the scikit-learn and XGBoost libraries, executed via the scripts `RFR_surrogate.py` and `XGBoost_surrogate.py`, and saved using joblib.

3.4.2 Error Metrics and Cross-Validation Strategy

Since reactor parameters involve very small fractional deviations, engineers use pcm (percent mille/parts per hundred thousand) to avoid working with tiny decimals (Mercier, 2025). It is expressed as:

$$pcm = \Delta k_{eff} \times 10^{-5} \tag{8}$$

Both ML training scripts utilised the function `nuclear_scoring_metric` to convert the RMSE to pcm, avoiding tiny decimals when interpreting results uncertainty.

K-fold cross validation is critical when training a limited dataset, to avoid overfitting (Tao *et al.*, 2025) and to provide an accurate prediction of how well the model generalises to an unseen dataset. K-fold validation maximises the usage of the small dataset by ensuring that all samples are used for both training and validation of the model, while avoiding the computational expense of exhaustive resampling methods (Bisong, 2019).

K-fold (with 5 folds) was implemented using scikitlearn for training the ML models by partitioning the dataset into five equal subsamples; training on four and validating on the fifth to ensure that every sample is utilised for one validation. This method ensures that the tree-based ML models do not overfit to the small dataset while also providing a necessary error metric for hyperparameter optimisation.

3.4.3 Hyperparameter optimisation

RMSE for the ML models was minimised using GridSearch; a hyperparameter tuning module used to optimise model performance by testing the combination of a range of values and picking the best hyperparameters (Bisong, 2019). This ensures that the trade-off between model bias and variance can be navigated successfully to prevent overfitting (Bisong, 2019). Bisong (2019) defines model bias as the error introduced by making simplified approximations of complex real-world problems (leading to underfitting), while model variance is the error introduced by the model's sensitivity to noisy data (leading to overfitting). The hyperparameters optimised using GridSearch for the RFR and XGBoost were:

Both RFR and XGBoost models:

- **`n_estimators`:** Dictates the total number of trees/boosting iterations used by each model (Breiman, 2001).
- **`max_depth`:** Sets the maximum depth of an individual tree. Restricting this limits model complexity, ensuring that it doesn't learn from overly complex interactions which leads to overfitting (Alizadeh, Allen and Mistree, 2020).

RFR model:

- **`max_features`:** The number of input variables selected to determine each node split. Forcing the algorithm to learn from a variety of feature combinations rather than the single dominant parameter (Tao *et al.*, 2025).
- **`min_samples_split`:** The minimum number of samples required to split a node. Increasing this prevents overfitting to noisy data (Alizadeh, Allen and Mistree, 2020).

XGBoost model:

- **`learning_rate`:** Scales the contribution of each newly added tree by a fraction between 0 and 1. A lower learning rate significantly reduces overfitting, though it requires proportionally larger **`n_estimators`** (Chen *et al.*, 2019).
- **`subsample`:** The fractional ratio of the sample space used to grow each tree. Setting this to a fraction forces the XGBoost to evaluate diverse subsets of the training data, preventing overfitting on small datasets (Chen *et al.*, 2019).

Table 4: Tested Hyperparameter Values for the 5-fold Cross-validation GridSearch.

ML model	Hyperparameter	GridSearch Hyperparameters		
RFR	`n_estimators`	100	200	300
	`max_depth`	None	10	20
	`max_features`	sqrt	log2	1.0
	`min_sample_split`	2	5	10
XGBoost	`n_estimators`	1000	1500	2000
	`max_depth`	3	4	5
	`learning_rate`	0.01	0.03	0.05
	`subsample`	0.5	0.65	0.8

Table 4 shows the hyperparameter matrix chosen for each ML model. The GridSearch algorithm finds the best combination of these parameters, finding the balance between bias and variance to produce the lowest possible uncertainty (RMSE). These values were chosen using the following strategies:

RFR:

The RFR relies heavily on the law of Large Numbers, meaning the algorithms generalisation error converges to a stable value as more trees are added (Breiman, 2001). Grid searching up to 300 `n_estimators` ensures that this convergence is reached without redundant computation. Additionally, the error of RFRs is dictated by the correlation between individual trees. Therefore, selecting random `max_features` minimises the correlation between trees, ultimately reducing generalisation error (Breiman, 2001). Including 1.0 allows the model to use all features to split

trees if it results in a reduced RMSE. Limiting `max_depth` and forcing higher `min_sample_split` limits the complexity of the RFR, avoiding overfitting to noise in the limited dataset (Breiman, 2001).

XGBoost:

Unlike the bagging architecture of RFRs, XGBoost uses boosting to reduce bias by learning from the errors of previous trees, making them highly susceptible to overfitting on small datasets (Chen and Guestrin, 2016). By limiting the learning rate to 0.05, the boosting process is forced to be conservative, preventing the model from overfitting to the sparse dataset (Chen *et al.*, 2019). Chen *et al.* (2019) explicitly states that a lower learning rate requires a larger number of estimators; allowing `n_estimators` to reach 2000 allows the boosting algorithm to account for the reduced learning rate. Furthermore, capping `max_depth` and `subsample` at low values limits tree complexity, ensuring that the XGBoost algorithm doesn't overfit to the noise in the limited dataset (Bisong, 2019).

3.4.4 Feature importance and interpretability

The Gini impurity index is an algorithm used in decision trees that selects the feature (e.g. fuel temperature) that yields the highest "information gain" and calculates the features contribution to predicting the target; known as Gini impurity, or feature importance (Bisong, 2019). By aggregating how much each feature improves each tree, the Gini index extracts a quantitative feature importance ranking that can be verified against physics, providing transparency in the ML models' predictions.

The Gini impurity index was used in both the RFR and XGBoost models to provide transparency in how each physical variable drives the model's prediction of k_{eff} (discussed in Sec. 3.4.1). The Gini importance of each model is visually analysed in Sec. 4.3.

3.5 Scope and Assumptions

For the scope of this project, the OpenMC outputs are treated as the absolute ground-truth. However, while extreme high-fidelity benchmarks aim for a standard deviation of 1 to 3 pcm, achieving this requires a massive number of simulated particles (Barrientos and Alfonso, 2021). Since MC uncertainty decreases at a rate of $1/\sqrt{N}$, pushing this uncertainty to this level would require exponentially more computation. The accepted mean standard deviation of 75.3 pcm

finds a balance between computation and predictive accuracy. Although, the presence of statistical error in the OpenMC ground-truth will pass into the ML surrogate models as noise, effecting their accuracy.

The ML training dataset was limited to 1000 samples due to the computational cost of the high-fidelity MC simulations (approximately 60 seconds per sample). While a larger dataset would improve model accuracy, this sample size provides sufficient diversity within the LHS boundaries to show a proof-of-concept for tree-based architectures in the simulation of SMRs.

The OpenMC ground-truth dataset was generated using a static solver, so did not consider time-dependent variables like fuel burnup. Therefore, the RFR and XGBoost models can only predict isolated instances of the SMR. Real-world SMRs are affected by dynamic, time-dependent parameters like fuel depletion/burnup (Mercier, 2025) and delayed neutrons (Zohuri, Balgehshiri and Lomonaco, 2025). However, predicting k_{eff} establishes the foundational viability of tree-based models before extending to computationally expensive depletion calculations.

The current model relies only on five input features (described in Sec. 3.2.2). While adding more features could theoretically improve model accuracy without significantly increasing prediction time, it comes with diminishing returns as each additional feature has less and less correlation with the target parameter (k_{eff}). Additionally, using more features will also have a diminishing effect on the interpretability of the tree-based models (Alizadeh, Allen and Mistree, 2020), making them harder to audit and consequently less compliant with the ALARP principles.

Because the exact design of the NPM is proprietary, some assumptions were made within the MIT ExaSMR benchmark to estimate the restricted designs (described in Sec. 3.2.1). Importantly, the control rod insertion steps were governed using a benchmark proxy (the BEAVRS benchmark) as suggested by Romano . These assumptions are acceptable for a computational proof of concept, as the fundamental physics of the MC model remain identical to the real-world. However, an operational model would require a perfect twin OpenMC model of the SMR.

Although tree-based models are excellent at interpolating predictions, their predictions become far less accurate when extrapolating data (Tao *et al.*, 2025). The scope of the surrogate models in this project is bound by the limits of the LHS, so any reactor states outside the training data cannot be accurately predicted. This problem was mitigated by extending the parameter boundaries beyond what would be expected during standard operation (Described in Sec. 3.2.2).

Despite the utilisation of Gini impurity for transparency, XGBoost algorithms are still fundamentally classed as “black boxes” (Chen *et al.*, 2019). While, a small number of shallow decision trees is easily interpretable, RFR and XGBoost rely on the aggregation of hundreds or

thousands of trees respectively. This architecture inherently obscures the underlying decision-making process, rendering the interpretability of the tree-based algorithms highly “questionable” (Alizadeh, Allen and Mistree, 2020, p. 284).